artifact. There is no need to rebuild the codebase to deploy an application to a specific environment: any aspect that changes between deployments (such as configuration) should be outside the application codebase.

2.1.2 API first

A cloud native system is usually made up of different services that communicate through APIs. Using an *API first* approach while designing a cloud native application encourages you to think about fitting it into a distributed system and favors the distribution of the work across different teams. By designing the API first, another team using that application as a backing service could create their solution against that API. By designing the contract up front, integration with other systems will be more robust and testable as part of the deployment pipeline. Internally, the API implementation can be changed without affecting other applications (and teams) depending on it.

2.1.3 Dependency management

All application dependencies should be declared explicitly in a manifest and be available for the dependency manager to download from a central repository. In the context of Java applications, we are usually well-equipped to follow this principle using tools like Maven or Gradle. The only implicit dependencies an application can have on the surrounding environment are the language runtime and the dependency manager tool. This means that private dependencies should be resolved via the dependency manager.

2.1.4 Design, build, release, run